BARUCH COLLEGE

Department of Mathematics • Master of Financial Engineering

DEEP LEARNING IN FINANCE | Lecture Series

Transformers for Financial Time Series Forecasting

Part II

Oualid Missaoui

Adjunct Faculty, Baruch College

Agenda

- Why transformers for time series?
- Time series forecasting: key challenges
- Direct (multi-horizon) forecasting with transformers
- Auto-regressive (AR) forecasting with transformers
- Core mechanisms: masking, teacher forcing, uncertainty
- Foundation Models for Time Series
- References

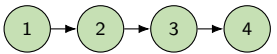
Why Transformers for Time Series?

From Part 1 we have **attention**; here is why it is a natural fit for forecasting — and where it improves on the RNN/LSTM/Seq2Seq models seen earlier.

- **Parallel over time:** no step-by-step recurrence — train on long histories fast (vs. sequential LSTM unrolling)
- **Long receptive field:** self-attention links any two timesteps in **one hop** — no vanishing-gradient decay over long lags
- **Native covariate fusion:** each token can carry target + covariates + static metadata
- **Scales to pretraining:** the same backbone absorbs billions of points $\Rightarrow$ **foundation models**

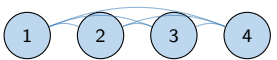
But time series $\neq$ language: continuous values, non-stationarity, and strict causality need care (patching, normalization, causal masks).

RNN



sequential path: distant steps are *many hops* apart

Attn



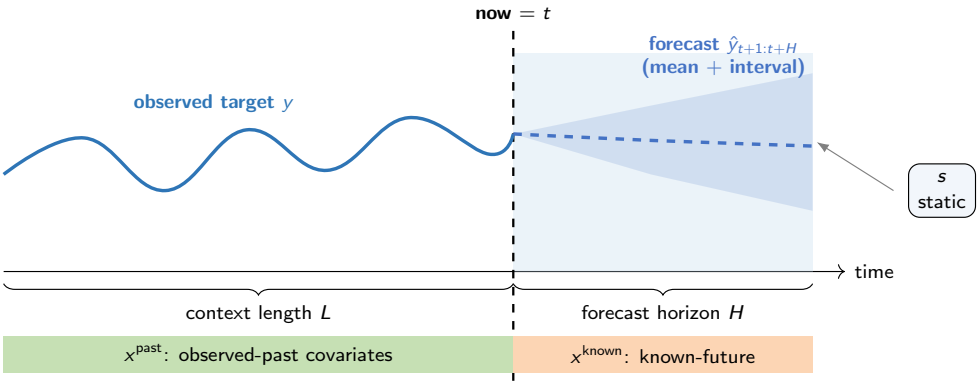
self-attention: *any-to-any*, constant path length

Time-Series Forecasting — Key Challenges

- **Nonstationarity & regime shifts:** changing trends/volatility (drift)
- **Seasonality & calendar effects:** multi-scale (daily/weekly/annual)
- **Long-range dependencies:** need wide receptive fields efficiently
- **Exogenous drivers:** static + known-future + observed-past covariates
- **Missing/irregular sampling & cold starts:** gaps, new series, short histories
- **Noise & outliers:** heavy tails; robust losses/likelihoods needed
- **Data leakage & evaluation:** strict causal splits; rolling-origin backtests
- **Scale mismatch:** per-series normalization (RevIN/instance norms)
- **Horizon alignment:** target vs covariate timing; avoid peeking
- **Uncertainty quantification:** intervals, quantiles, proper scoring rules

▶ **In finance:** regime shifts = volatility clustering; known-future = FOMC/earnings/holidays; static = sector/ticker; **RevIN** normalizes each window on input and de-normalizes the forecast to absorb level/scale drift.

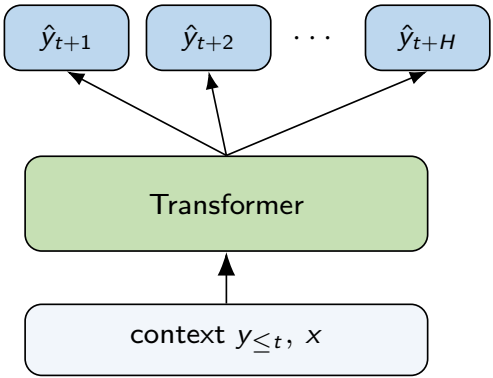
The Forecasting Setup at a Glance



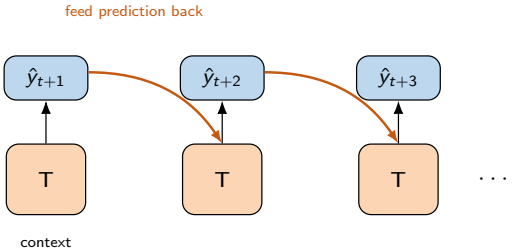
▶ **In finance:** y = returns/vol; x^{past} = realized vol & order-flow; x^{known} = scheduled events (FOMC, earnings, expiries); s = sector/ticker.

Two Paradigms at a Glance: Direct vs Autoregressive

Direct (multi-horizon)



Autoregressive (AR)



H sequential steps; each output is fed back in.

One forward pass $\rightarrow$ **all** H steps in parallel.

Same backbone, different **decoding**: parallel one-shot vs. recursive rollout. Everything that follows flows from this choice.

Direct Forecasting — Setup & Notation

Goal. Learn a single mapping from a past context to the entire horizon **in one shot**:

$$\hat{\boldsymbol{y}}_{t+1:t+H} = f_\theta\Big(\boldsymbol{y}_{t-L+1:t},\ \boldsymbol{x}^{\text{past}}_{t-L+1:t},\ \boldsymbol{x}^{\text{known}}_{t+1:t+H},\ \boldsymbol{s}\Big)$$

Notation

- $\boldsymbol{y}$: target series; (L) context length; (H) forecast horizon
- $\boldsymbol{x}^{\text{past}}$: observed-past covariates (lags, rolling stats, sensors)
- $\boldsymbol{x}^{\text{known}}$: **known-future** covariates (calendar, tariffs, promos)
- $\boldsymbol{s}$: static features (entity metadata)

Data windows & batching

- Each example = **(past window, future window)** $\rightarrow$ H labels
- Slide the window over time/entities to build the batch
- Maintain **causality**: no future-target leakage into inputs

▶ **In finance:** context = a trailing window of returns/vol; horizon H = the multi-day path you actually trade on.

Direct Forecasting — Architecture & Loss

Architecture pattern (Transformer)

- Encoder (or encoder–decoder) over past tokens + **future tokens** (one per step or per patch)
- **Masking:** future tokens attend the past + each other, **never** true future targets

  (the **block** mask — see *Attention Masking*)

Losses (horizon-aware)

$$\mathcal{L}(\theta) = \sum_{k=1}^{H} \ell\big(\hat{y}_{t+k}, y_{t+k}\big) \quad \text{(MSE/MAE/Pinball/CRPS)}$$

Refinements

- Optional **joint** head (multivariate Gaussian / low-rank Σ) $\rightarrow$ models step-to-step correlations
- **Horizon weights** w_k emphasize near- vs far-term steps
- Per-step vs. joint distribution drives how intervals are sampled (see *Probabilistic Forecasting*)

Point losses (MSE/MAE) vs. probabilistic (Pinball/CRPS). CRPS: Continuous ranked probability score.

Direct Forecasting — Inference & Uncertainty

Inference (testing)

$$\hat{\boldsymbol{y}}_{t+1:t+H} \leftarrow f_\theta\Big(\boldsymbol{y}_{t-L+1:t},\ \boldsymbol{x}^{\text{past}}_{t-L+1:t},\ \boldsymbol{x}^{\text{known}}_{t+1:t+H},\ \boldsymbol{s}\Big)$$

- **One forward pass → all H steps** (low latency, long horizons)
- **Streaming:** update context each tick; recompute once per forecast time

Known-future covariates at test time

- Provide full $\boldsymbol{x}^{\text{known}}_{t+1:t+H}$ (calendar, scheduled events)
- If partially unknown: fill known parts; use scenarios/fallback for the rest

Uncertainty (see *Probabilistic Forecasting*)

1. **Per-step:** independent quantiles/likelihoods (simple, fast)
2. **Joint:** correlated horizon head; sample trajectories via $\epsilon \sim \mathcal{N}(0, \Sigma)$

Joint Σ captures that adjacent horizon errors move together — essential for realistic fan charts.

Direct Forecasting — Evaluation & Pitfalls

Evaluation protocol

- **Backtest:** rolling origin (walk-forward) — for each cut t, predict $t+1{:}t+H$ in one shot
- **Report:** horizon-averaged metrics **and** per-step breakdowns (RMSE/Pinball by k)
- **Fairness:** compare **latency/throughput** vs. AR, not just accuracy

Common pitfalls (and fixes)

- **Leakage** from preprocessing $\rightarrow$ fit scalers/encoders **inside** each train window
- **Misaligned covariates** $\rightarrow$ ensure $\boldsymbol{x}^{\text{known}}$ lines up with $t+1{:}t+H$
- **Far-horizon overfit** $\rightarrow$ horizon weights, dropout, shrink joint Σ

▶ **In finance:** always evaluate with rolling-origin / walk-forward and fit scalers *inside* each window — leakage inflates backtested Sharpe.

AR (DGM) Forecasting — Setup & Teacher Forcing

Goal. Learn the one-step transition, then roll forward (chain-rule factorization):

$$p_\theta\left(\boldsymbol{y}_{t+1:t+H} \mid \mathcal{H}_t\right) = \prod_{k=1}^{H} p_\theta\left(y_{t+k} \mid \mathcal{H}_t,\ y_{t+1:t+k-1},\ \boldsymbol{x}_{t+1:t+k}\right)$$

Notation

- $\mathcal{H}_t$: observed history up to t (targets + covariates)
- $\boldsymbol{x}$: covariates (past observed + possibly **known-future** at each step)

Training batches (teacher forcing)

- Input: past window; **feed ground-truth** y (see *Teacher Forcing*)
- Target: **next step** y_{t+1}
- Shift over time/entities → batches of (context → y_{t+1})

DGM: Deep Generative Model. Multi-step quality *emerges* from many one-step objectives.

AR (DGM) Forecasting — Architecture & Loss

Architecture & masking (Transformer as DGM)

- **Decoder-only** or encoder–decoder with a **strict causal mask** along time
- Each query (for $t+1$) attends to tokens $\leq t$ only

  (the **causal** mask — see *Attention Masking*)
- Probabilistic head $\rightarrow$ a distribution (Gaussian / quantiles / discrete tokens)

Losses (next-step likelihood/quantile)

$$\mathcal{L}(\theta) = -\log p_\theta\left(y_{t+1} \mid \boldsymbol{y}_{t-L+1:t}, \boldsymbol{x}_{t-L+1:t+1}\right)$$

(or Pinball/CRPS for quantiles)

The head choice fixes the **tokenization** of value: continuous (Gaussian/quantile) vs. discrete (Chronos-style) — a key axis for foundation models.

DGM Autoregressive (AR) Forecasting — Testing

Rollout (testing)

1. Warm-start with observed history $\boldsymbol{y}_{t-L+1:t}$.
2. For $k = 1..H$:

$$\hat{y}_{t+k} \sim p_\theta\big(\cdot \mid \hat{\boldsymbol{y}}_{t-L+1:t+k-1},\ \boldsymbol{x}_{t+1:t+k}\big)$$

   Append $\hat{y}_{t+k}$ to the context and continue.

- **Latency/compute:** cost scales with H (one decode per step, unless using block decoding)
- **Streaming-friendly:** emit one step per tick; no need to re-forecast the full horizon each time

Uncertainty

- Naturally **trajectory-level**: sample multiple rollouts (MC) from p_θ to get fan charts/intervals
- Distributional heads (quantiles/MoG/tokenized) yield calibrated per-step predictive distributions

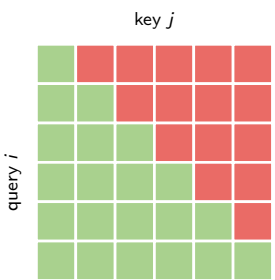
Known-future covariates at test time

- Thread $\boldsymbol{x}_{t+k}$ into **each step** of the rollout
- If partially known: scenario trees (branch per covariate scenario), or condition only on observed drivers

The Mechanism Behind the Split: Attention Masking

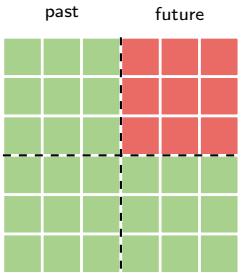
A mask decides, for each **query** (row), which **keys** (columns) it may attend to. allowed blocked

AR: strict causal mask



position i sees only $j \leq i$ (no peeking ahead)

Direct: block / future-bidirectional

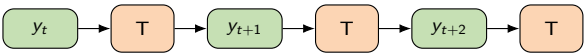


future tokens attend past + each other (**cross-horizon**); target *values* are never keys

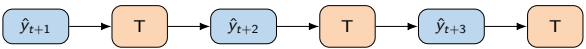
Causal ⇒ recursive rollout (AR). Block ⇒ one-shot parallel horizon (Direct).

Teacher Forcing, Exposure Bias & Error Accumulation

Training (teacher forcing): feed the ground-truth previous value.

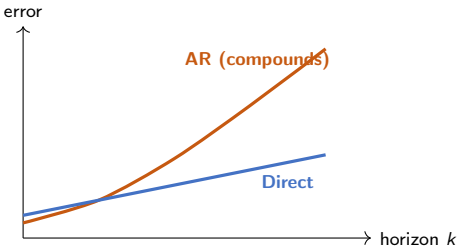


Inference (free-running): feed the model's own prediction — errors feed forward.



Exposure bias: train-time inputs (clean) $\neq$ test-time inputs (noisy).
Fixes: scheduled sampling, noise injection, block/direct decoding.

Error vs. horizon



AR errors **recursively compound**; Direct optimizes the whole horizon jointly, so far-step error grows more slowly.

Probabilistic Forecasting: Quantiles, Pinball & CRPS

Point forecasts are not enough in finance — we need **calibrated intervals**.

Pinball (quantile) loss for level τ:

$$\rho_\tau(y, \hat{q}) = \begin{cases} \tau\,(y - \hat{q}), & y \geq \hat{q} \\ (1-\tau)(\hat{q} - y), & y < \hat{q} \end{cases}$$

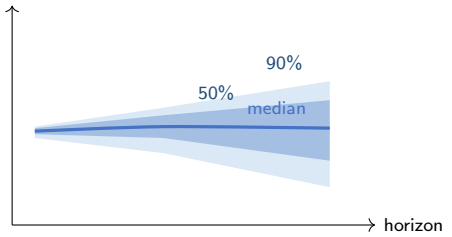
- Asymmetric: penalizes under- vs over-prediction differently $\Rightarrow$ estimates the τ-quantile
- Train a few levels (e.g. $0.1, 0.5, 0.9$) for an interval

CRPS (proper score, whole distribution):

$$\mathrm{CRPS}(F, y) = \int_{-\infty}^{\infty} \big(F(z) - \mathbf{1}\{z \geq y\}\big)^2 dz$$

$= \int_0^1 2\rho_\tau \, d\tau$ — integrates pinball over all levels.

Predictive fan chart



Quantile bands = VaR-style risk; proper scoring rules reward **calibration**, not just accuracy.

Direct vs AR — Modeling

Aspect	Direct (Multi-Horizon)	Autoregressive (AR / DGM)
Objective	Learn $f_\theta : (\text{past}, \text{cov}^{\text{past}}, \text{cov}^{\text{known}}_{1:H}) \to \hat{\boldsymbol{y}}_{1:H}$ in **one shot**	Learn $p_\theta(y_{t+1} \mid \text{history})$; get H steps by **rolling forward**
Factorization	No chain rule at inference; optimize joint horizon loss	Chain rule $p(\boldsymbol{y}_{1:H}) = \prod_{k=1}^{H} p(y_k \mid y_{<k}, \cdot)$
Architecture & Masking	Encoder or enc-dec; **future tokens** query past; block access to true future targets; allow **cross-horizon** attention	Decoder-only or enc-dec; **strict causal mask** along time; each step attends to past + generated outputs
Batches & Labels	Each sample yields H labels (past→future window)	Each sample yields **1-step** label (teacher forcing: past $\to y_{t+1}$)
Loss	Horizon-aware (MSE/MAE/Pinball/CRPS), optionally **joint** multivariate head	Next-step NLL/quantile; multi-step quality emerges from rollout
Error dynamics	No recursive compounding; horizon consistency from joint training	**Exposure bias** and **error accumulation** across steps
Uncertainty	Per-step or **joint** horizon distributions (correlated errors) in one pass	Natural **trajectory sampling** via stochastic rollout; per-step distributions

Direct vs AR — Deployment & Practice

Aspect	Direct (Multi-Horizon)	Autoregressive (AR / DGM)
Inference	**One forward pass** $\rightarrow$ all H (low latency for long horizons)	H decoding steps (or block decode); cost scales with horizon
Known-future covariates	Ingest **entire** $\boldsymbol{x}_{1:H}^{\text{known}}$ cleanly in one pass	Must **thread** $\boldsymbol{x}_{t+k}$ into **each** step of rollout
Compute & Latency	Fixed per forecast event; great when H large	Scales with H; can be memory-light per step
Streaming / Online	Recompute full H each tick (can be heavier)	**Streaming-friendly**: emit one step at a time
Regularization / Tricks	Horizon weights, dropout on future tokens, temporal smoothing, patching	Scheduled sampling, block decoding (DirRec/MIMO), noise on inputs, distillation
Evaluation	Rolling-origin; report horizon-avg + per-step metrics; include **latency/throughput**	Rolling-origin; report per-step + aggregated metrics; plot **error growth vs horizon**
Use when...	Long horizons, reliable **known-future** drivers, need **low latency** multi-step	Unknown future drivers, **streaming**, generative multi-path forecasts, short–mid horizons

Foundation Models for Time Series

A **foundation model** is pretrained once on massive, diverse time-series corpora, then forecasts **unseen** series with little or no fine-tuning (**zero/few-shot**).

What is shared

- One backbone, many domains → transfer across series
- Forecast off-the-shelf — no per-dataset training
- Usually **probabilistic** (intervals, not just points)

Design axes to watch (how they differ)

- **Tokenization:** continuous patches vs. **quantized** value tokens ("language of time series")
- **Decoding:** Direct one-shot vs. Autoregressive rollout
- **Scope:** univariate vs. any-variate (multivariate, covariates)

The four we examine

- **Moirai** (Salesforce) — patches, Direct, any-variate
- **Lag-Llama** (Meta FAIR) — lag tokens, AR
- **Chronos** (Amazon) — quantized tokens, AR
- **TimesFM** (Google DeepMind) — patches, Direct

▶ **In finance:** zero-shot backbones let you forecast a *new* ticker or instrument with no history of its own — valuable for cold-start assets.

“Patching” in Time-series Transformers

Idea: Instead of feeding every time point as a token, **group contiguous segments of length P into a single token (a patch)** — just like Vision Transformers treat image patches as tokens.

Input representation

- Original series:

$$[x_1, x_2, x_3, \cdots, x_T]$$

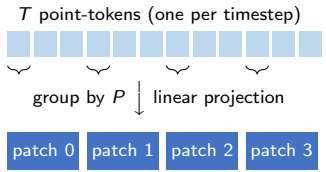
- Divide into non-overlapping or overlapping windows (patches):

$$[x_1 \cdots x_P],\ [x_{P+1} \cdots x_{2P}], \cdots$$

- Each patch $\rightarrow$ **linear projection** $\rightarrow$ **embedding vector**

Transformer attention

- Self-attention operates **across patches**, not individual timesteps
- Captures *long-range temporal dependencies* efficiently
- Reduces sequence length from T to $T/P \rightarrow$ lower memory & computation



Sequence length $T \rightarrow T/P$; patches may overlap.

Moirai (Salesforce, 2024)

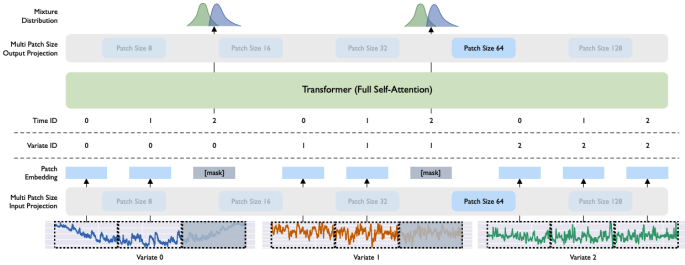


Figure 2. Overall architecture of MOIRAI. Visualized is a 3-variate time series, where variates 0 and 1 are target variables (i.e. to be forecasted, and variate 2 is a dynamic covariate (values in forecast horizon known). Based on a patch size of 64, each variate is patchified into 3 tokens. The patch embeddings along with sequence and variate id are fed into the Transformer. The shaded patches represent the forecast horizon to be forecasted, whose corresponding output representations are mapped into the mixture distribution parameters.

What to notice

- **Idea:** one **any-variate** forecaster — multiple series/covariates share the Transformer
- **Tokenization: multi patch-size** projections ($8 \ldots 128$) pick the right resolution
- **Decoding: Direct** — masked future patches predicted in one pass
- **Output:** a **mixture distribution** (probabilistic), not a point
- **Novel:** sequence & variate IDs let attention span time *and* variables

Gerald Woo et al. (2024), Unified Training of Universal Time Series Forecasting Transformers.

Lag-Llama (Meta FAIR, 2024)

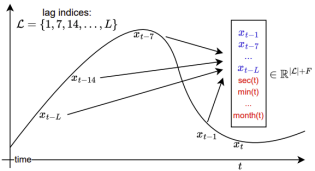


Figure 1: For a time series, we depict the tokenization at the timestep t of the value x_t which contains lag features constructed using an example set of lag indices $\mathcal{L}$, where each value in the vector is from the *past* of x_t (in blue), and F possible temporal covariates (date-time features) constructed from timestamp t (red).

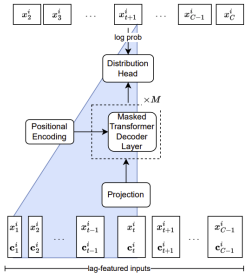


Figure 2: The `Lag-Llama` architecture. `Lag-Llama` learns to output a distribution over the values of the next time step based on lagged input features. The input to the model is the token of a univariate time series i at a given timestep, $\mathbf{x}_t^i$, constructed as described in Sec.4.1. Here, we use $\mathbf{c}_t^i$ to refer to all additional covariates used along with the value at a timestep t, which include the $|\mathcal{L}|$ lags, F date-time features, and summary statistics. The inputs are projected through M masked decoder layers. The features are then passed through the distribution head and trained to predict the parameters of the forecast distribution of the next timestep.

What to notice

- **Idea:** a LLaMA-style **decoder-only** LM for univariate series
- **Tokenization:** each token = **lag features** $(x_{t-1}, x_{t-7}, \ldots)$ + date-time covariates
- **Decoding: Autoregressive** — next-step distribution, roll forward
- **Output:** probabilistic head (sample trajectories → intervals)
- **Novel:** lags make a single scalar step information-rich

Kashif Rasul et al. (2024), Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting.

Chronos (Amazon, 2024)

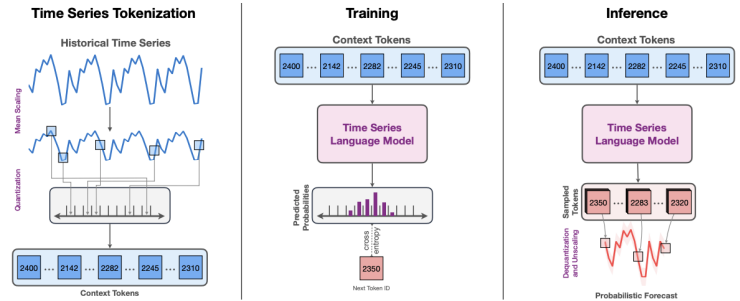


Figure 1: High-level depiction of Chronos. (**Left**) The input time series is scaled and quantized to obtain a sequence of tokens. (**Center**) The tokens are fed into a language model which may either be an encoder-decoder or a decoder-only model. The model is trained using the cross-entropy loss. (**Right**) During inference, we autoregressively sample tokens from the model and map them back to numerical values. Multiple trajectories are sampled to obtain a predictive distribution.

- **Idea:** treat forecasting as **language modeling** of time series
- **Tokenization:** scale, then **quantize** values into a fixed vocabulary (bins → token IDs)
- **Decoding: AR** next-token prediction with cross-entropy — an off-the-shelf LM
- **Novel:** sample many token trajectories → calibrated predictive distribution

Abdul Fatir Ansari et al. (2024), Chronos: Learning the Language of Time Series.

TimesFM (Google DeepMind, 2024)

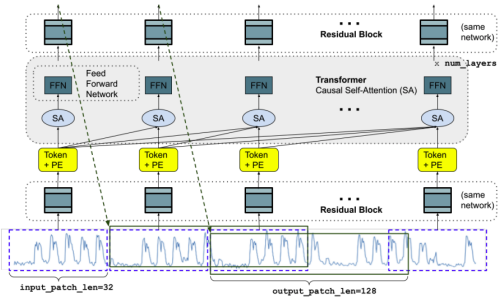


Figure 1: We provide an illustration of the TimesFM model architecture during training, where we show a input time-series of a specific length that can be broken down into input patches. Each patch along is processed into a vector by a residual block (as defined in the model definition) to the model dimension of the transformer layers. The vector is then added to positional encodings and fed into n_l stacked transformer layers. SA refers to self-attention (note that we use multi-head causal attention) and FFN is the fully connected layer in the transformer. The output tokens are then mapped through a residual block to an output of size `output_patch_len`, which is the forecast for the time window following the last input patch seen by the model so far.

What to notice

- **Idea:** a **decoder-only** patched foundation model for forecasting
- **Tokenization:** input series → **patches**; each patch is one token
- **Decoding:** **Direct**-style — emits a **longer output patch/block** per step
- **Novel:** input/output patch-length **asymmetry** ⇒ few steps for long horizons
- **Scale:** pretrained on a massive real-world TS corpus

Abhimanyu Das et al. (2024), A decoder-only foundation model for time series forecasting.

Comparing Time-Series Foundation Models

Aspect	Moirai (Salesforce, 2024)	Lag-Llama (Meta FAIR, 2024)	Chronos (Amazon, 2024)	TimesFM (Google DeepMind, 2024)
Architecture	Patch **encoder–decoder** Transformer; patch projection + cross-time/variable attention (any-variate, multi-freq)	**Decoder-only** causal Transformer over **lag tokens** (LM-style)	**Decoder-only** causal Transformer over **quantized value tokens** (tokenizer + embedding)	**Patch encoder + long-patch decoder** (encoder–decoder); ViT-like patches with cross-attention
Paradigm	**Direct** multi-horizon	**Autoregressive**	**Autoregressive**	**Direct** multi-horizon
Pretraining scope	Large multi-domain TS corpora (billions of points)	LM-style self-supervised pretraining on broad TS	Generative pretraining across many public TS datasets	Massive real-world TS corpus (hundreds of millions/billions of points)
Inference style	**One-shot**: predict full horizon at once	**Rollout**: step-by-step (can sample trajectories)	**Rollout**: step-by-step (trajectory sampling)	**One-shot** via **output patches/blocks**
Typical strengths	Strong cross-domain transfer; handles multivariate & mixed frequencies	Streaming, zero/few-shot generative use; flexible tasks beyond forecasting	Calibrated generative forecasting; multi-path sampling	Fast long-horizon forecasts; strong zero/few-shot transfer

References

- Siva Rama Krishna Kottapalli et al. (2025), *Foundation Models for Time Series: A Survey*
- Gerald Woo et al. (2024), *Unified Training of Universal Time Series Forecasting Transformers*
- Abhimanyu Das et al. (2024), *A decoder-only foundation model for time series forecasting*
- Abdul Fatir Ansari et al. (2024), *Chronos: Learning the Language of Time Series*
- Kashif Rasul et al. (2024), *Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting*